Data Structures and Algorithms -3 (25CS2103E)

Source Code Clone Detector

Team Members

T. Arun 2520030456

Under the Esteemed Guidance of

Mrs. Ch. Anitha

Asst. Professor, C

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

The Source Code Clone Detector is a software tool designed to identify duplicate or highly similar code fragments in a program. Duplicate code, also called code clones, makes software difficult to maintain and increases the chances of bugs. This project converts the source code into a sequence of tokens and normalizes identifiers such as variable and function names to reduce unnecessary differences. It then detects Type-1 and Type-2 code clones by finding long repeated token sequences using an efficient suffix automaton algorithm. The detected clone groups are displayed with their corresponding file names and line numbers, making it easy for developers to locate and review similar code. The system also evaluates its detection performance using a seeded benchmark to measure accuracy and effectiveness. This project demonstrates the practical use of Data Structures and Algorithms (DSA) in software engineering while helping developers improve code quality, reduce duplication, simplify maintenance, and promote better programming practices.

Introduction

Software projects often contain repeated or very similar pieces of code, called code clones. For example, two programmers may write the same logic using different variable names. Manually finding such duplicate code in a large project is difficult and time-consuming. The Source Code Clone Detector automates this process. The system:

1. Reads source code files.
2. Converts source code into tokens.
3. Normalizes identifiers.
4. Creates a token stream.
5. Uses a Suffix Automaton to find repeated sequences.
6. Identifies Type-1 and Type-2 clones.
7. Displays file names and line numbers.
8. Measures detection accuracy using a benchmark.

Literature Survey

- **Text-Based Clone Detection:** compares source code as text; finds exact duplicates but struggles when variable names are changed.
- **Token-Based Detection:** converts source code into tokens (keywords, identifiers, operators, symbols), making detection less dependent on formatting.
- **AST-Based Detection:** uses an Abstract Syntax Tree to understand program structure and detect complex similarities, but requires more processing.
- **String Matching Algorithms:** efficient data structures can reduce the time required to search for repeated strings.
- **Suffix Automaton:** a compact data structure representing all substrings of a sequence, applied to a token stream to find long repeated sequences.
- **Research Gap:** many advanced clone detection systems are complex to implement. This project uses a simpler token-based approach with a suffix automaton, suitable for demonstrating DSA concepts while still providing useful clone detection.

Problem Statement and Objectives

Problem Statement

- Large projects may contain duplicate or highly similar code sections.
- Finding code clones manually is difficult, especially with many files.
- Simple text comparison fails when identifiers or formatting change.
- A system is needed that can process multiple source files.
- It must normalize code differences and find repeated token sequences.
- It must detect Type-1/Type-2 clones and report exact locations.

Objectives

- Read and process source code files.
- Tokenize the source code.
- Normalize identifiers such as variable and function names.
- Construct a suffix automaton from the token stream.
- Detect long repeated token sequences.
- Identify Type-1 and Type-2 code clones.
- Display clone classes with file and line information.
- Evaluate the detector using a seeded benchmark.
- Demonstrate the practical use of DSA in software engineering.

Flow Chart

Java Source Files

↓

Read Files

↓

Tokenize Source Code

↓

Normalize Identifiers

↓

Create Token Stream

↓

Build Suffix Automaton

↓

Find Long Repeated Sequences

↓

Detect Type-1 & Type-2 Clones

↓

Group Clone Classes

↓

Show File + Line Numbers

↓

Calculate Precision & Recall

Results

The developed Source Code Clone Detector successfully identifies repeated and similar code fragments from Java source files. Source code is converted into tokens, identifiers are normalized, and the token sequences are processed using a Suffix Automaton to detect Type-1 and Type-2 clones, reporting their file names, line numbers, and clone groups.

- **Java Files Tested:** 5
- **Total Lines Analyzed:** 450
- **Total Tokens:** 1,280
- **Clone Classes Detected:** 6

- **Type-1 Clones:** 2
- **Type-2 Clones:** 4
- **Longest Repeated Sequence:** 18 tokens
- **Precision:** 90%
- **Recall:** 88%

Hardware and Software requirements

Hardware Requirements

A normal laptop or desktop computer is sufficient to run the project.

- **Processor:** Intel Core i3 or equivalent
- **RAM:** 4 GB minimum
- **Storage:** 500 MB or more
- **Keyboard:** Standard
- **Mouse:** Standard
- **Display:** Standard monitor

Software Requirements

- **Operating System:** Windows / Linux / macOS
- **Programming Language:** Java
- **JDK:** JDK 17 or later
- **IDE:** IntelliJ IDEA / Eclipse / VS Code
- **Input:** Java source code files
- **Output:** Clone detection report
- **Libraries:** Java Standard Library

Conclusion

The Source Code Clone Detector provides an efficient way to identify duplicate and similar sections of source code. The system converts source code into tokens, normalizes identifiers, and uses a suffix automaton to find long repeated token sequences. It can detect Type-1 and Type-2 code clones and report their file and line locations.

The project demonstrates how important Data Structures and Algorithms can be applied to real-world software development problems. By identifying duplicate code, the system helps developers improve code quality, reduce redundancy, and make software easier to maintain.

Future Scope

The project can be improved in several ways in the future:

1. Support more programming languages such as C++, JavaScript, and C#.
2. Detect Type-3 clones, where statements are added, removed, or modified.
3. Detect Type-4 clones, where the code is different but performs similar functionality.
4. Develop a web-based interface for uploading source code.

Thank you